full adder cell of the PPU placed in the $(i-1)^{th}$ row. The other input of PPU_4/PPU_6 is the AND/NAND gate output. In the structure of PPU_5, the two inputs of the full adder are fed from the partial products generated by two NAND gates. This PPU is applied in the $(n-1)^{th}$ row and $(2n-2)^{th}$ column. PPU_7 cells are placed in the last column of 2nd to $(n-2)^{th}$ rows, in which two inputs result from partial products generated by an AND gate and a NAND gate. The last PPU, PPU_8, is located in $\left(\frac{n}{2}+1\right)^{th}$ column and n^{th} row of an n-bit signed array multiplier. One of this PPU's inputs is the constant bit '1' [27], and the other input is the *Sum* output of the full adder (PPU_4), which is located in $(n-1)^{th}$ row and $\left(\frac{n}{2}+1\right)^{th}$ column. Table 12 lists the number of applied PPU_1–PPU_8 cells and their computational steps in the proposed n-bit signed CSA-based array multiplier.

Table 12: Characteristics of the proposed CSA-based signed array multiplier.

Combinational Block	No. of Applied Blocks	No. of Computational Steps
PPU_1	$n-2$	18
PPU_2	1	18
PPU_3	1	2
PPU_4	n^2-5n+7	25
PPU_5	1	28
PPU_6	$n-2$	25
PPU_7	$n-3$	28
PPU_8	1	9
Full Adder [23]	$n-3$	22
AND gate	1	5

Similar to the serial unsigned array multiplier, the method of overlapping repetitive serial steps has been applied to reduce the computational steps and energy consumption of the n-bit signed CSA-based array multiplier in serial architecture. The structures of PPU_3 and PPU_8, containing one constant input with a value of '1', have also been redefined to reduce these criteria further. PPU_1 and PPU_4 of the n-bit signed array multiplier are the same as the unsigned array multiplier's PPU_1 and PPU_2, respectively. Hence, repeated serial steps can be overlapped in the signed array multiplier's PPU_1 and PPU_4, the same as the ones in the unsigned array multiplier. The number of computational steps required to implement PPU_1 and PPU_4 in the serial signed CSA-based array multiplier is reduced to 18 (18% improvement) and 25 (7% improvement), respectively. The implementation algorithm of PPUs' constituent building blocks, including NAND gate (see Table 13), AND gate (see (1) and Table 6), half adder (see (2), (3), and Table 7) and full adder (see (6) and (7) and Table 9) has been investigated in detail to apply the overlapping repetitive serial steps method to reduce the computational steps of each PPU, resulting in a reduction of computational steps and energy consumption of the proposed signed array multiplier.

Table 13: Serial IMPLY-based implementation of the NAND gate.

Step	Operation	Equivalent Logic
1	$S_1 = 0$	FALSE (S_1)
2	$(b \rightarrow 0) \equiv (b \rightarrow S_1) = S_1{}'$	$S_1{}' = \bar{b}$
3	$(a \rightarrow S_1{}') = S_1{}''$	$S_1{}'' = \overline{a.b} = \boldsymbol{NAND(a,b)}$

The NAND gate's output is computed in three computational steps and stored in a work memristor based on its serial implementation algorithm. The NAND gate's output (the half adder and full adder cells' input in $PPU_{2,5,6,7}$) is inverted once, according to (2), (3), (6), and (7) and Tables 7 and 9. Hence, there are no repetitive

serial steps to be overlapped between the NAND gate and the half adder and full adder cells. Conversely, the last two computational steps of the AND gate are repeated as the first two steps of the half adder and full adder's algorithm and are overlapped in PPU_2 and PPU_7. The proposed PPU_2 and PPU_7 implementation algorithms are shown in Tables 14 and 15, respectively, based on (13)–(16).

$$\mathbf{PPU_2}:\ Sum_{HA} = ab \oplus \overline{cd} \equiv \left[\overline{(a \to \bar{b})} \to (c \to \bar{d})\right] \to \overline{\left[(a \to \bar{b}) \to \overline{(c \to \bar{d})}\right]} \tag{13}$$

$$Cout_{HA} = ab \cdot \overline{cd} \equiv \overline{\left[(c \to \bar{d}) \to (a \to \bar{b})\right]} \tag{14}$$

$$\mathbf{PPU_7}:\ Sum_{FA} = \overline{ab} \oplus cd \oplus C_{in} \equiv \left[\left(\overline{(a \to \bar{b})} \to \overline{(c \to \bar{d})}\right) \to \left(\left((a \to \bar{b}) \to (c \to \bar{d})\right) \to C_{in}\right)\right]$$
$$\to \overline{\left[\left(\overline{\left(\overline{(a \to \bar{b})} \to \overline{(c \to \bar{d})}\right)} \to \overline{\left((a \to \bar{b}) \to (c \to \bar{d})\right)}\right) \to \overline{C_{in}}\right]} \tag{15}$$

$$Cout_{FA} = \overline{ab} \cdot cd + C_{in}(\overline{ab} + cd)$$
$$\equiv \overline{\left[\left(\overline{(a \to \bar{b})} \to \overline{(c \to \bar{d})}\right) \to \overline{\left(\left(\left((a \to \bar{b}) \to (c \to \bar{d})\right) \to C_{in}\right)\right)}\right]} \tag{16}$$

The computational steps of the proposed PPU_2 and PPU_7 equal 18 steps (10% improvement) and 28 steps (7% improvement), respectively, reduced by two steps compared to their classic circuits (20 and 30 steps). The number of memristors required for these two proposed PPUs has not changed compared to the classic structures.

Table 14: The implementation algorithm of the proposed PPU_2 in the signed array multiplier.

Step	Operation	Equivalent Logic
1	$S_1 = 0$	$FALSE(S_1)$
2	$S_2 = 0$	$FALSE(S_2)$
3	$(b \to 0) \equiv (b \to S_1) = S_1'$	$S_1' = \bar{b}$
4	$(a \to S_1') = S_1''$	$S_1'' = \bar{a} + \bar{b} = \bar{\alpha}$
5	$(d \to 0) \equiv (d \to S_2) = S_2'$	$S_2' = \bar{d}$
6	$(c \to S_2') = S_2''$	$S_2'' = \bar{c} + \bar{d} = \beta$
7	$S_3 = 0$	$FALSE(S_3)$
8	$S_4 = 0$	$FALSE(S_4)$
9	$(S_1'' \to 0) \equiv (S_1'' \to S_3) = S_3'$	$S_3' = ab = \alpha$
10	$(S_2'' \to 0) \equiv (S_2'' \to S_4) = S_4'$	$S_4' = cd = \bar{\beta}$
11	$(S_1'' \to S_4') = S_4''$	$S_4'' = ab + cd = \alpha + \bar{\beta}$
12	$(S_2'' \to S_1'') = S_1'''$	$S_1''' = cd + (\bar{a} + \bar{b}) = \bar{\beta} + \bar{\alpha}$
13	$(S_3' \to S_2'') = S_2'''$	$S_2''' = (\bar{a} + \bar{b}) + (\bar{c} + \bar{d}) = \bar{\alpha} + \beta$
14	$S_3 = 0$	$FALSE(S_3)$
15	$(S_4'' \to 0) \equiv (S_4'' \to S_3) = S_3'$	$S_3' = (\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d}) = \bar{\alpha} \cdot \beta$
16	$(S_2''' \to S_3') = S_3''$	$S_3'' = (ab \cdot cd) + \left((\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d})\right) = \alpha \oplus \beta = \boldsymbol{Sum}$
17	$S_2 = 0$	$FALSE(S_2)$
18	$(S_1''' \to 0) \equiv (S_1''' \to S_2) = S_2'$	$S_2' = (\bar{c} + \bar{d}) \cdot ab = \beta \cdot \alpha = \boldsymbol{C_{out}}$

Table 15: The implementation algorithm of the proposed PPU_7 in the signed array multiplier.

Step	Operation	Equivalent Logic
1	$S_1 = 0$	$FALSE(S_1)$
2	$S_2 = 0$	$FALSE(S_2)$
3	$S_3 = 0$	$FALSE(S_3)$
4	$S_4 = 0$	$FALSE(S_4)$
5	$(b \to 0) \equiv (b \to S_1) = S_1{}'$	$S_1{}' = \bar{b}$
6	$(a \to S_1{}') = S_1{}''$	$S_1{}'' = \bar{a} + \bar{b} = \alpha$
7	$(d \to 0) \equiv (d \to S_2) = S_2{}'$	$S_2{}' = \bar{d}$
8	$(c \to S_2{}') = S_2{}''$	$S_2{}'' = \bar{c} + \bar{d} = \bar{\beta}$
9	$(S_1{}'' \to 0) \equiv (S_1{}'' \to S_3) = S_3{}'$	$S_3{}' = ab = \bar{\alpha}$
10	$(S_2{}'' \to 0) \equiv (S_2{}'' \to S_4) = S_4{}'$	$S_4{}' = cd = \beta$
11	$(S_3{}' \to S_4{}') = S_4{}''$	$S_4{}'' = (\bar{a} + \bar{b}) + cd = \alpha + \beta$
12	$(S_1{}'' \to S_2{}'') = S_2{}'''$	$S_2{}''' = ab + (\bar{c} + \bar{d}) = \bar{\alpha} + \bar{\beta}$
13	$S_1 = 0$	$FALSE(S_1)$
14	$(S_2{}''' \to 0) \equiv (S_2{}''' \to S_1) = S_1{}'$	$S_1{}' = (\bar{a} + \bar{b}) \cdot cd = \alpha \cdot \beta$
15	$(S_4{}'' \to S_1{}') = S_1{}''$	$S_1{}'' = ab \cdot (\bar{c} + \bar{d}) + (\bar{a} + \bar{b}) \cdot cd = \bar{\alpha} \cdot \bar{\beta} + \alpha \cdot \beta$
16	$S_3 = 0$	$FALSE(S_3)$
17	$(C_{in} \to 0) \equiv (C_{in} \to S_3) = S_3{}'$	$S_3{}' = \overline{C_{in}}$
18	$(S_1{}'' \to S_3{}') = S_3{}''$	$S_3{}'' = \left((\bar{a} + \bar{b}) + cd\right) \cdot \left(ab + (\bar{c} + \bar{d})\right) + \overline{C_{in}}$ $= (\alpha + \beta) \cdot (\bar{\alpha} + \bar{\beta}) + \overline{C_{in}}$
19	$S_1 = 0$	$FALSE(S_1)$
20	$(S_3{}'' \to 0) \equiv (S_3{}'' \to S_1) = S_1{}'$	$S_1{}' = \left(ab \cdot (\bar{c} + \bar{d}) + (\bar{a} + \bar{b}) \cdot cd\right) \cdot C_{in}$ $= (\bar{\alpha} \cdot \bar{\beta} + \alpha \cdot \beta) \cdot C_{in}$
21	$(S_2{}''' \to C_{in}) = C_{in}{}'$	$C_{in}{}' = (\bar{a} + \bar{b}) \cdot cd + C_{in} = \alpha \cdot \beta + C_{in}$
22	$S_2 = 0$	$FALSE(S_2)$
23	$(C_{in}{}' \to 0) \equiv (C_{in}{}' \to S_2) = S_2{}'$	$S_2{}' = \left(ab + (\bar{c} + \bar{d})\right) \cdot \overline{C_{in}} = (\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}$
24	$(S_4{}'' \to S_2{}') = S_2{}''$	$S_2{}'' = ab \cdot (\bar{c} + \bar{d}) + \left(\left(ab + (\bar{c} + \bar{d})\right) \cdot \overline{C_{in}}\right)$ $= \bar{\alpha} \cdot \bar{\beta} + \left((\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}\right)$
25	$(S_4{}'' \to C_{in}{}') = C_{in}{}''$	$C_{in}{}'' = ab \cdot (\bar{c} + \bar{d}) + \left((\bar{a} + \bar{b}) \cdot cd + C_{in}\right)$ $= \bar{\alpha} \cdot \bar{\beta} + (\alpha \cdot \beta + C_{in})$
26	$(C_{in}{}'' \to S_1{}') = S_1{}''$	$S_1{}'' = \left[(\alpha + \beta) \cdot \left((\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}\right)\right] + (\overline{\alpha \oplus \beta} \cdot C_{in})$ $= \alpha \oplus \beta \oplus C_{in} = \boldsymbol{Sum}$
27	$S_3 = 0$	$FALSE(S_3)$
28	$(S_2{}'' \to S_3) = S_3{}'$	$S_3{}' = (\alpha + \beta) \cdot (\alpha \cdot \beta + C_{in})$ $= \alpha \cdot \beta + C_{in}(\alpha + \beta) = \boldsymbol{C_{out}}$